Power Grid Simulation

README

1. Project Overview

The Power Grid Visualizer is a C++ application that simulates a hierarchical power distribution network with real-time load balancing, demand propagation, and capacity optimization using Raylib. It demonstrates advanced graph algorithms for infrastructure resilience and interactive network visualization.

Key Features:

- Three-tier network topology (Power Plants → Substations → Consumers)
- Widest-path algorithm for optimal load rerouting
- Real-time demand propagation and throttling mechanisms
- Interactive visualization with zoom-based LOD rendering
- Console logging for detailed network analysis

2. Core Data Structures

2.1 Graph Components

Vertex_A (Infrastructure Nodes):

- **node_type**: "power_plant" or "substation"
- **max_limit**: Maximum capacity (kW)
- **current_downstream_demand**: Actual load (kW)
- **x, y**: 2D coordinates

Vertex_B (Consumer Nodes):

- **node_type**: "residential" | "hospital" | "industrial" | "institute"
- **demand**: Current consumption (kW)
- **x, y**: 2D coordinates

Edge (Network Connections):

- **loss**: Transmission loss (0.5–5 kW depending on type)
- **max_load**: Line capacity (kW)
- **current_load**: Present power flow (kW)

2.2 Network Topology

Three-tier hierarchy:

1. Tier 1 (40% substations): Connected to power plants
2. Tier 2 (60% substations): Connected to prior substations
3. Consumers: Distributed proportionally (50% residential, 20% hospital, 20% institute, 10% industrial)

Capacity initialization:

- Substations: max_limit = 1.5 × total_downstream_demand
- Power plants: max_limit = 1.2 × sum(connected_substation_limits)

3. Key Algorithms

3.1 Widest Path Algorithm

Modified Dijkstra maximizing bottleneck capacity instead of cost:

State: (max_capacity, path)
Priority Queue: MAX-heap on capacity

```
1. Initialize source with infinity capacity
2. For each neighbor:
   new_capacity = min(current_capacity, edge_available_capacity)
   if new_capacity &gt; 0 and better than previous:
       Add to queue
3. Return path with highest capacity to target
```

Complexity: O((V+E) log V)

Features: Bidirectional exploration, cycle detection, handles both forward/backward edges

3.2 Load Balancing Pipeline

Step 1: Overload Detection

```
For each edge:
    if (current_load / max_load ≥ 0.75):
        Add to problems_to_fix
Sort by severity ratio (descending)
```

Step 2: Path Finding

- Find highest-capacity path from source to target
- Calculate required reduction: current_load − (0.70 × max_load)

Step 3: Validation Rules

Rule	Condition	Action
20% Rule	rerouted_load < 0.20 × required_reduction	Throttle instead
75% Rule	Any edge in path would exceed 75% utilization	Throttle instead
Feasibility	Both rules pass	Apply rerouting

Step 4: Action

```
If rerouting viable:
    original_edge.current_load -= rerouted_load
    For each path_edge:
        path_edge.current_load += rerouted_load
Else:
    Add target node to throttling queue
```

3.3 Demand Propagation

Upward cascade when consumer demand increases:

```
consumer_delta_demand = +10 kW
    ↓
substation.current_demand += 10 kW
    ↓
For each parent:
    proportion = parent_edge.max_load / total_incoming_capacity
    parent_delta = 10 × proportion
    Recursively update parent
```

Prevents loops via visited set

3.4 Throttling (Demand Reduction)

Proportional brownout when infrastructure insufficient:

```
For each throttled node:
    total_demand = sum(connected_consumers.demand)
    For each consumer:
        proportion = consumer.demand / total_demand
        consumer.demand -= reduction × proportion
```

4. User Interface

4.1 Controls

Key	Action	Result
[I]	Increase random demand	Consumer +20% (min 5 kW), propagate up, zoom to event
[O]	Fix overload	Run load balancing, show path (orange→green) or throttle
[U]	Upgrade node	Selected substation +25% capacity, propagate upstream
[C]	Clear highlights	Remove visualization overlays
[Mouse Wheel]	Zoom	0.1×–3.0× with LOD changes
[Left Drag]	Pan	Move camera target
[Left Click]	Select	Choose node/edge, update inspector
[BACKSPACE]	Menu	Return to input, clear graph

4.2 Visualization

Colors:

- Power Plants: PINK (15px)
- Substations: BLUE (10px)
- Residential: BROWN (5px)
- Hospital: DARKBLUE (7px)
- Industrial: ORANGE (7px)
- Institute: PURPLE (7px)

Edge States:

- Normal: Yellow (plant→sub), SkyBlue (sub→sub), LightGray (sub→consumer)
- Overloaded: RED pulsing arrows (10 Hz animation)
- Fixed edge: ORANGE arrows (0.5s highlight)
- Solution path: GREEN thick arrows with direction

LOD Rendering:

- zoom ≥ 1.0x → All nodes visible
- 0.5x ≤ zoom < 1.0x → Hide consumers
- zoom < 0.5x → Show only power plants

4.3 Inspector Panel

Shows selected node/edge info:

```
--- SELECTED NODE ---
Type: substation
STATUS: NODE OVERLOADED [if applicable]
Current Load: XXX kW
Max Limit: XXX kW
Press 'U' to upgrade
```

5. Compilation & Usage

5.1 Build Requirements

- g++ compiler
- MSYS2 development environment
- Raylib graphics library
- Standard C++ libraries: iostream, vector, unordered_map, queue, cmath, random, iomanip

5.2 Input Configuration

- Number of power plants
- Number of substations
- Total consumers
- Base demand per consumer type (kW)

5.3 Simulation Interface

- **Input Screen**: Configure network parameters
- **Visualization Screen**: Use interactive controls to manage network
- **Console Logging**: Enable detailed debugging output by redirecting console to file

5.4 Global Variables

```
int resident_demand, hospital_demand,
    industry_demand, institute_demand;
random_device rd;
mt19937 g(rd());  // Random number generator
```

Simulation Controls:

- Press [I] to trigger demand events
- Press [O] to run load balancing
- Press [U] to upgrade selected nodes
- Press [C] to clear visualization highlights
- Press [BACKSPACE] to return to menu
- Use mouse controls for navigation

6. Performance & Design Insights

6.1 Optimization Techniques

1. **Spatial Grid** – O(1) collision detection via 50×50px cells
2. **LOD Rendering** – Hide consumers at low zoom levels
3. **Single Fix/Frame** – One overload fix prevents cascades
4. **Bidirectional Edges** – Alternative routes via adj_reverse_power

6.2 Algorithm Choices

Design Choice	Why?
Widest Path	Prioritizes capacity over distance (throughput > latency for power)
±30% Variance	Realistic heterogeneous demand distribution
Proportional Throttling	Fair demand shedding across all consumers
Two-rule Validation	Prevents cascading failures
Bidirectional Edges	Models smart grid redundancy

6.3 Scalability

- 100+ substations
- Up to 9000 consumers
- 10000+ edges
- ~60 FPS performance
- 200–400 MB memory usage

7. Event Simulation Examples

7.1 Random Demand Event ([I] key)

Tested efficiently with:

1. Select random consumer
2. Check if local edge has capacity
3. Increase demand by max(5, 20% of current)
4. Propagate change upward recursively
5. Zoom camera to event, highlight node

Console Output:

```
--- Random Event ---
Increased demand for residential (0x...)
  By: 5.0 kW. New Total: 15.0 kW.
  Parent Substation: (0x...)
```

7.2 Load Balancing ([O] key)

Successful rerouting:

```
--- Load Balancing Action ---
Overloaded Edge: (0x... → 0x...)
   Original Load: 150 / 200 (75%)
   Path Capacity: 45.00 kW
   ACTUAL Rerouting: 20.00 kW
   New Load: 130 / 200
   Via Path: 0x... → (0x... | 120 kW) → ...
   Status: OK (Fixed)
```

Throttling fallback:

```
Load Balancing: No alternative paths exist.
   → SCHEDULING DEMAND REDUCTION for node: 0x... by 20.00 kW
--- Applying Throttling ---
   Throttled node 0x.... Reduced by: -10.50 kW
```

7.3 Upgrade Capacity ([U] key)

Node upgrade and propagation:

1. Select a substation node
2. Press [U] to increase capacity by 25%
3. Propagate increase upstream to parents
4. Update all connecting edges' max_load
5. Recalculate network capacity limits

7.4 Clear Highlights ([C] key)

Reset visualization state:

1. Clear all overload highlights
2. Remove path visualization arrows
3. Clear throttled node indicators
4. Reset to default rendering mode

7.5 Navigation Controls

Mouse interactions:

- **Mouse Wheel**: Zoom in/out (0.1×–3.0×) with automatic LOD adjustment
- **Left Click + Drag**: Pan camera to different network areas
- **Left Click on Node/Edge**: Select and display information in inspector panel

8. Spatial Layout Algorithm

BFS-based positioning with grid optimization:

Power Plants (roots):

- Random placement [0, 2000] × [0, 2000]
- Min separation: 400px

Substations (BFS traversal):

- Near parent: ±200px offset
- Min separation: 50px

Consumers (leaf nodes):

- Near substation: ±50px offset
- Min separation: 5px

Grid collision detection (50×50px cells):

- Check only 3×3 neighboring cells
- O(1) average case vs $O(n^2)$ naive

9. Key Formulas

Power loss by edge type:

- Plant → Substation: 2–5 kW (long distance)
- Substation → Substation: 1–3 kW (medium distance)
- Substation → Consumer: 0.5–1 kW (short distance)

Capacity initialization:

- Substation: 1.5 × Σ(consumer demands)
- Power plant: 1.2 × Σ(substation limits)

Overload threshold:

```
if (current_load / max_load ≥ 0.75) → FLAG AS OVERLOADED
Target state: current_load ≤ 0.70 × max_load
```

Available capacity on edge:

```
available = edge.max_load − edge.current_load
```

10. Potential Enhancements

Node/Edge Repair: Implement a "failure" state where edges that remain overloaded for prolonged periods "break" (become unusable) and must be manually "repaired" by the user, introducing resource management and maintenance scheduling elements to the simulation.

Variable Demand: Replace user-triggered demand events with realistic 24-hour "demand curves" for each consumer type, causing overloads to appear and resolve organically throughout the simulation cycle without manual intervention.

Smarter Throttling: Extend the nodes_to_throttle map to track individual throttling amounts per node rather than applying uniform reductions, enabling fine-grained demand shedding that reflects actual grid operator strategies and consumer priorities.

11. Project Contributions

11.1 Function Distribution & Group Contributions

The project functions were strategically distributed among four team groups based on functional dependencies and UI components:

11.1.1 Data Mapping & Distribution (Keshav Swami & Harry Singh)

This group handled the core data structure mappings and distribution logic essential for network initialization:

- **Distributor**: Distribution system logic for power flow initialization
- **mapping**: General mapping functionality for node and edge relationships
- **substation_consumer_mapping**: Mapping relationships between substations and consumers
- **power_plant_mapping**: Mapping of power plants to network components

11.1.2 Spatial Layout & Position Validation (Parv Dixit & Partyaksh Tyagi)

This group developed algorithms for spatial organization and geometric validation of network nodes:

- **Layout graph**: Graph structure and organization for spatial representation
- **get_distance**: Distance calculation utility for spatial computations
- **isposition_okay_A**: Position validation function A for node placement verification
- **isposition_okay_B**: Position validation function B for alternate validation logic

11.1.3 Load Balancing & Core Network Optimization (All Team Members)

This cross-functional group implemented the critical load balancing and capacity management algorithms:

- **overloading_edge**: Edge overload detection and management
- **fix_overloading_problem**: Overloading resolution algorithm
- **increase_random_demand**: Dynamic demand generation and simulation
- **propagate_limit_increase**: Capacity limit propagation across network
- **can_handle_increase**: Capacity verification for demand increases
- **try_add_capacity_path**: Attempt to add paths with sufficient capacity
- **find_highest_capacity_path**: Algorithm to identify optimal capacity paths
- **add_realistic_connections**: Network connection establishment logic

11.1.4 Demand Management & Capacity Upgrades (Harry Singh & Keshav Swami)

This group focused on demand reduction, throttling, and node capacity management:

- **check_node_overloads**: Node-level overload verification
- **apply_demand_reduction_updates**: Demand management and reduction logic
- **upgrade_selected_node_limit**: Node capacity upgrade functionality

11.1.5 User Interface & Visualization

State Input Interface (Keshav Swami & Harry Singh - 50% each):

- Handles user input for network parameters and simulation controls
- Manages state initialization and configuration

State Visualization Interface (Parv Dixit & Partyaksh Tyagi - 50% each):

- Renders network visualization and real-time updates
- Displays load distribution and system status

11.2 Individual Contributions

11.2.1 Keshav Swami (B24EE1033)

Backend Components:

- **vertex_A, vertex_B**: Core vertex data structures
- **distributor**: Distribution system logic
- **substation_consumer_mapping**: Mapping relationships between substations and consumers
- **overloading_edge**: Edge overload detection and management
- **overloading_info**: Information retrieval for overloading scenarios
- **find_highest_capacity_path**: Algorithm to identify optimal capacity paths
- **check_node_overloads**: Node-level overload verification

UI Development:

- **State Input Case**: 50% responsibility

11.2.2 Harry Singh (B24EE1101)

Backend Components:

- **edge**: Core edge data structures
- **mapping**: General mapping functionality
- **power_plant_mapping**: Mapping of power plants to network components
- **increase_random_demand**: Dynamic demand generation and simulation
- **apply_demand_reduction_updates**: Demand management and reduction logic
- **can_handle_increase**: Capacity verification for demand increases
- **upgrade_selected_node_limit**: Node capacity upgrade functionality

UI Development:

- **State Input Case**: 50% responsibility

11.2.3 Parv Dixit (B24EE1093)

Backend Components:

- **Graph Layout**: Graph structure and organization
- **isposition_okay_B**: Position validation function B
- **Hash Pair**: Hash pair data structure implementation
- **fix_overloading_problem**: Overloading resolution algorithm
- **propagate_demand_change**: Demand propagation throughout network

- **try_add_capacity_path**: Attempt to add paths with sufficient capacity

UI Development:

- **State Visualization Case**: 50% responsibility

11.2.4 Partyaksh Tyagi (B24CM1088)

Backend Components:

- **isposition_okay_A**: Position validation function A
- **get_distance & get_random**: Distance calculation and random value generation utilities
- **add_realistic_connections**: Network connection establishment logic
- **find_edge**: Edge identification and retrieval
- **propagate_limit_increase**: Capacity limit propagation across network
- **PathState**: Path state management and tracking

UI Development:

- **State Visualization Case**: 50% responsibility

11.3 Responsibility Allocation Summary

Team Member	Backend Functions	UI Component	UI Load
Keshav Swami (B24EE1033)	7	State Input	50%
Harry Singh (B24EE1101)	7	State Input	50%
Parv Dixit (B24EE1093)	6	State Visualization	50%
Partyaksh Tyagi (B24CM1088)	6	State Visualization	50%

11.4 Contribution Notes

- Each team member has been assigned specific backend functionality related to graph processing, demand management, and network optimization.